Lab#2



– Digital Electronics –

Sequential Circuits

This purpose of this lab is practicing with sequential circuits. For complex circuits, it is often necessary to combine both combinational and sequential circuits.

Contents:

1. n-bit synchronous counter
2. Pseudo-random generator
3. Programmable flashing digits

Abbreviations and acronyms:

IC – Integrated Circuit
LE – Logic Element
LED – Light Emitting Diode
LUT – Look Up Table
MUX – Multiplexer
RTL – Register Transfer Level

1 – n-bit synchronous counter

1. **Write an HDL file** that describes a parametric n-bit synchronous counter (with asynchronous reset) use a sequential always/process and implement the counting operation by using the HDL statement:

```
Q <= Q + 1;
```

In order to allow the addition of `unsigned` numbers, you need to properly manage data types. For VHDL remember to add the *numeric_std* package of the IEEE library:

```
USE ieee.numeric_std.all;
```

Equip your counter with a synchronous clear signal, a synchronous enable and a synchronous signal to count up/down. For down counting you can use:

```
Q <= Q - 1;
```

Finally, add a synchronous load signal and an input signal to initialize the value of the counter.

The final counter looks like the one in Figure 1 and the behavior is shown in Figure 2. Pay attention to the priority of each input and remember that **a positive-edge triggered synchronous sequential circuit must have in sensitivity list only the clock and the asynchronous reset**.

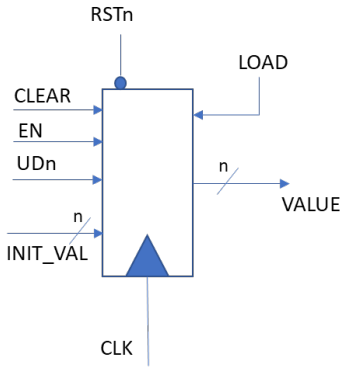


Figure 1 – An n-bit synchronous counter.

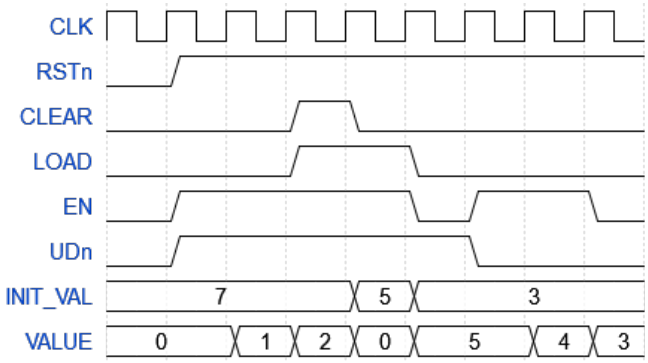


Figure 2 – n-bit synchronous counter timing diagram

2. Ask an AI to generate the same functionality you implemented (with the same HDL you employed) and compare the result
3. **Fix n=16, then simulate your circuit** to verify its correctness.
4. Open the RTL viewer (`Tools -> Netlist viewers -> RTL viewer`) How many logic elements (LEs) are used to implement your circuit configured as a 16-bit counter ? What is the maximum frequency, *fmax*, at which your circuit can be operated? (see Appendix A for details on finding *fmax*).

2 – Pseudo-random generator

Design and implement a circuit that turns on/off the 10 LEDs according with the Linear Feedback Shift Register (LFSR) shown in Figure 3. KEY(0) acts as the *asynchronous reset* and forces all the flip-flops in the circuit to logic '0'. Each pattern generated by the LFSR is kept stable for two seconds. Initialize all the flip-flops of the LFSR (by pressing KEY(1)) with the pattern defined with the switches (SW). Use button KEY(2) to drive a T-Flip-Flop in order to enable/disable the system (at the reset the system is disabled, then pressing KEY(2) the system is enabled, pressing again KEY(2) the system is disabled and so on). Note: the pattern all zeros is keeping the LFSR in a "zero state", so it is not much interesting. Use a counter (from point 1) and a comparator (from laboratory 1) and determine the amount of time you have to wait before generating a new pattern with the LFSR. The whole circuit receives the 50 MHz clock signal provided on the DE1 board (`CLOCK_50` input signal, to be managed as a clock input coming from the external world). **Do not derive any other clock signals in your design; make sure that all the flip-flops in your circuit are clocked directly by the 50 MHz clock signal, of course you can use the enable signal of each flip-flop**.

You have to perform the following steps:

1. **Draw a block scheme** of the circuit.
2. **Draw the timing diagram** of the circuit.
3. **Describe the circuit** in HDL.
4. Ask an AI to generate the same functionality you implemented (with the same HDL you employed) and compare the result
5. **Simulate the circuit** to verify its correctness. **Suggestion**: make your design parametric with respect to the amount of time you have to wait such that you can easily simulate it by setting the wait time to a low value. Then, you change the parameter which regulates the wait time and you can proceed with the design flow.
6. **Implement your circuit** on the DE1-SoC board, check the RTL view and compare it with your block scheme. Test its functionality by looking at the LEDs. Check that your design can run at least at 50 MHz (remember to create the sdc file and to read it).
7. **Estimate the dynamic power consumption** of the circuit, as explained in Appendix B, by simulating it for at least 20 ms with 50 MHz clock frequency in two different realistic conditions. Then, compare the results.

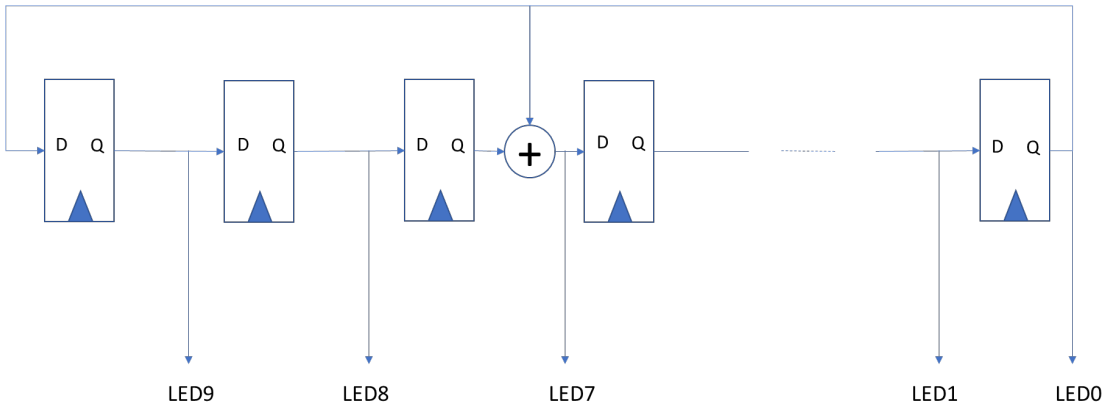


Figure 3 – LFSR

3 – Programmable Timer

Design and implement a circuit that acts as a timer with programmable starting value. The starting value is expressed in minutes by setting the switches SW(2:0) according with the following table.

SW(2:0)	Starting time [min]
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Table I: starting time in minutes

At the power up the starting value must be zero. The timer exploits 3 buttons: KEY(0), KEY(1) and KEY(2) as follows:

- KEY(0) is the asynchronous reset, which resets all the circuit and puts the starting value to zero;
- KEY(1) loads the starting value defined by SW(2:0);
- KEY(2) starts the timer down counting from the starting value to zero. If the starting value is zero pressing KEY(2) has no effect.

The timer displays on HEX(2) the minutes and on HEX(1) and HEX(0) the seconds. If the starting value is different from zero, when the timer reaches zero it makes LED0 blink (1 second on – 1 second off) for 5 times (10 seconds).

All the sequential elements in the circuit receive the 50 MHz clock signal provided on the DE1-SoC board (`CLOCK_50` input signal, to be managed as a clock input coming from the external world). **Do not derive any other clock signals in your design; make sure that all the flip-flops in your circuit are clocked directly by the 50 MHz clock signal, of course you can use the enable signal of each flip-flop**.

Suggestion: use counters (from point 1) and comparators (from laboratory 1) to design the programmable timer.

You have to perform the following steps:

1. **Draw a block scheme** of the circuit.
2. **Draw the timing diagram** of the circuit.
3. **Describe the circuit** in HDL.
4. Ask an AI to generate the same functionality you implemented (with the same HDL you employed) and compare the result
8. **Simulate the circuit** to verify its correctness. **Suggestion**: make your design parametric with respect to the amount of time you have to wait such that you can easily simulate it by setting the wait time to a low value. Then, you change the parameter which regulates the wait time and you can proceed with the design flow.
5. **Implement your circuit** on the DE1-SoC board, check the RTL view and compare it with your block scheme. Test its functionality by operating the implemented switches. Check that your design can run at least at 50 MHz (remember to create the sdc file and to read it).
6. **Estimate the dynamic power consumption** of the circuit, as explained in Appendix B, by simulating it for at least 20 ms with 50 MHz clock frequency in two different realistic conditions. Then, compare the results.

Appendix A – Introduction to TimeQuest Timing Analyzer

This section demonstrates how to set up timing constraints and obtain timing information for a logic circuit using TimeQuest Timing Analyzer. For more details please refer to Intel-Altera document "*Using TimeQuest Timing Analyzer*".

Timing analysis is a process of analyzing delays in a logic circuit to determine the conditions under which the circuit operates reliably. One example of a timing analysis computation is to find the maximum clock frequency for a circuit including combinational components and flip-flops. To operate correctly, the clock period has to be long enough to accommodate the delay on the longest path in the circuit. Computing the longest delays in a circuit and comparing these delays to the clock period is a basic function of a timing analyzer.
The timing analyzer can be used to guide computer-aided design (CAD) tools in the implementation of logic circuits. By placing timing constraints on the maximum clock frequency, it is possible to direct the CAD tools to seek an implementation that meets those constraints.

Design Example

As an example we will use an adder that adds three 8-bit numbers and produces a sum output. The inputs are A, B, and C, which are stored in registers reg_A, reg_B and reg_C at the positive edge of the clock. The three registers provide inputs to the adder, whose result is stored in the reg_sum register. The output of the reg_sum register drives the output port sum. The diagram of the circuit is shown in Figure 1A.

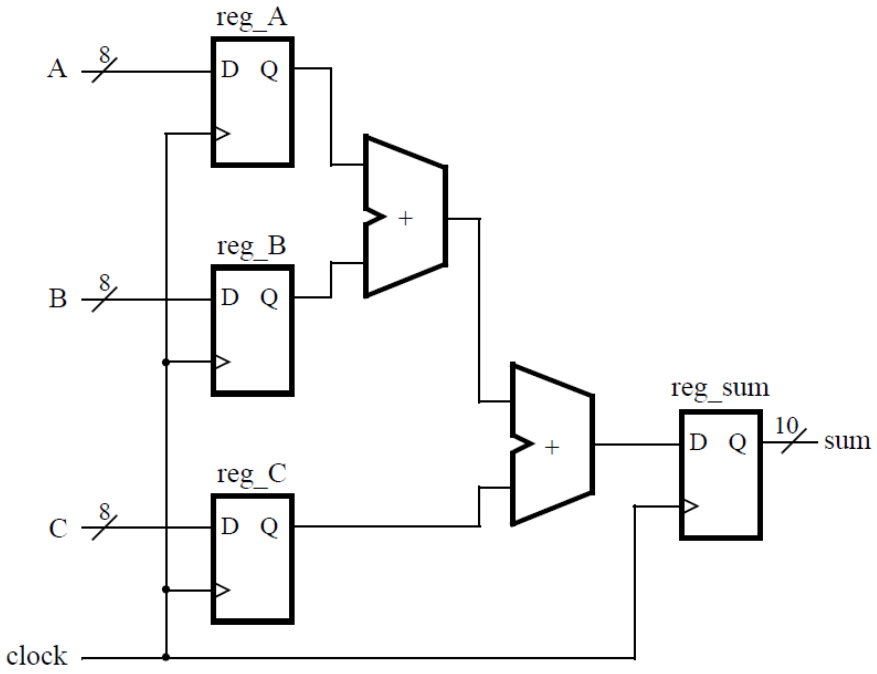


Figure 1A: Example design.

The HDL source code for the design is given below. Note that the **preserve**/*synthesis keep* comment is included in this code. This comment is interpreted as a directive that instructs the Quartus Prime software to retain the specified nodes in the final implementation of the circuit and keep their names as stated. This directive will allow us to refer to these nodes in the tutorial.

```
// Verilog
module add_three_numbers (
  input clk,
  input [7:0] A, B, C,
  output [9:0] sum
);

  (*preserve*)  reg [7:0] reg_A, reg_B, reg_C;
  (*preserve*)  reg [9:0] sum_reg;
  wire [9:0] sum_i;

  always @(posedge clk) begin
    reg_A <= A;
    reg_B <= B;
    reg_C <= C;
    sum_reg <= sum_i;
  end

  assign sum_i = {2'b00, reg_A} + {2'b00, reg_B} + {2'b00, reg_C};
  assign sum = sum_reg;

endmodule

-- VHDL
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY add_three_numbers IS
PORT ( clock : IN STD_LOGIC;
     A, B, C : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
         sum : OUT STD_LOGIC_VECTOR(9 DOWNTO 0));
END add_three_numbers;

ARCHITECTURE Behavior OF add_three_numbers IS
-- Registers
SIGNAL reg_A, reg_B, reg_C : UNSIGNED(7 DOWNTO 0);
SIGNAL sum_i : UNSIGNED(9 DOWNTO 0);
SIGNAL sum_reg : STD_LOGIC_VECTOR(9 DOWNTO 0);

ATTRIBUTE keep : boolean;
```

```
ATTRIBUTE keep OF reg_A, reg_B, reg_C, sum_reg : SIGNAL IS true;

BEGIN
  PROCESS ( clock )
  BEGIN
    IF (clock'EVENT AND clock = '1') THEN
      reg_A <= UNSIGNED(A);
      reg_B <= UNSIGNED(B);
      reg_C <= UNSIGNED(C);
      sum_reg <= STD_LOGIC_VECTOR(sum_i);
    END IF;
  END PROCESS;

  sum_i <= ("00" & reg_A) + ("00" & reg_B) + ("00" & reg_C);

  sum <= sum_reg;

END Behavior;
```

To begin the tutorial create a new Quartus Prime project for the design of our example circuit. Select as the target device the usual FPGA chip of the DE1-SoC board. Type the HDL code above into a file and add this file to the project.
You do not need to make any pin assignments for this example. Compile the project to see the results of timing analysis. These results will be available in the compilation report, once the design is compiled.

Using TimeQuest

Open the *Timequest Timing Analyzer* section of the Compilation Report, and click on the *Clocks* item to select it. In the Clocks display panel that opens on the right-hand side of the Quartus Prime window, notice that the clock signal from the example design has been given a clock period constraint of 1 ns (frequency of 1000 MHz). This is a default constraint that the Quartus Prime CAD tool places on any clock signal in a design project that does not have any user-provided timing constraints. Right-click on the name of the clock signal and select the command *Report Timing ...* (in *Timequest UI*). This action opens the Report Timing dialog shown in Figure 2A.

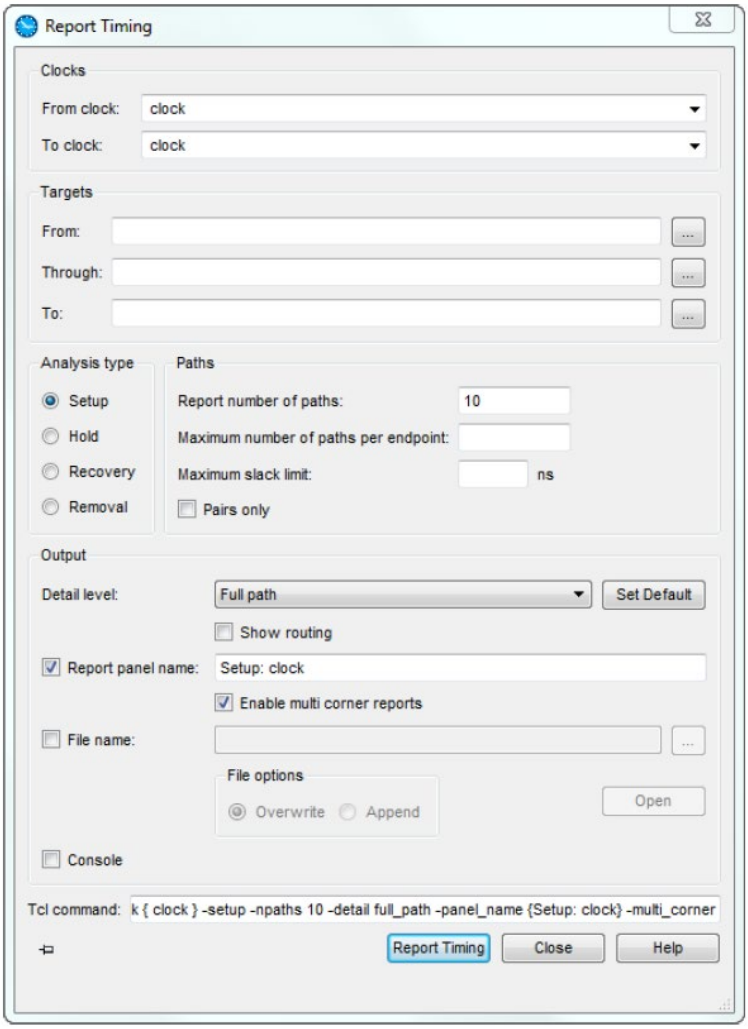


Figure 2A: Report Timing dialog.

Click the drop-down arrow in the *From clock* item and select the clock signal. This selection is used to instruct TimeQuest to analyze all paths in the example circuit that start and end at flip-flops that are clocked by the clock signal. The various settings are displayed in this window: accept all of the default selections and click on the *Report Timing* button. This command opens the *Timequest Graphical User Interface* (GUI) shown in Figure 3A.

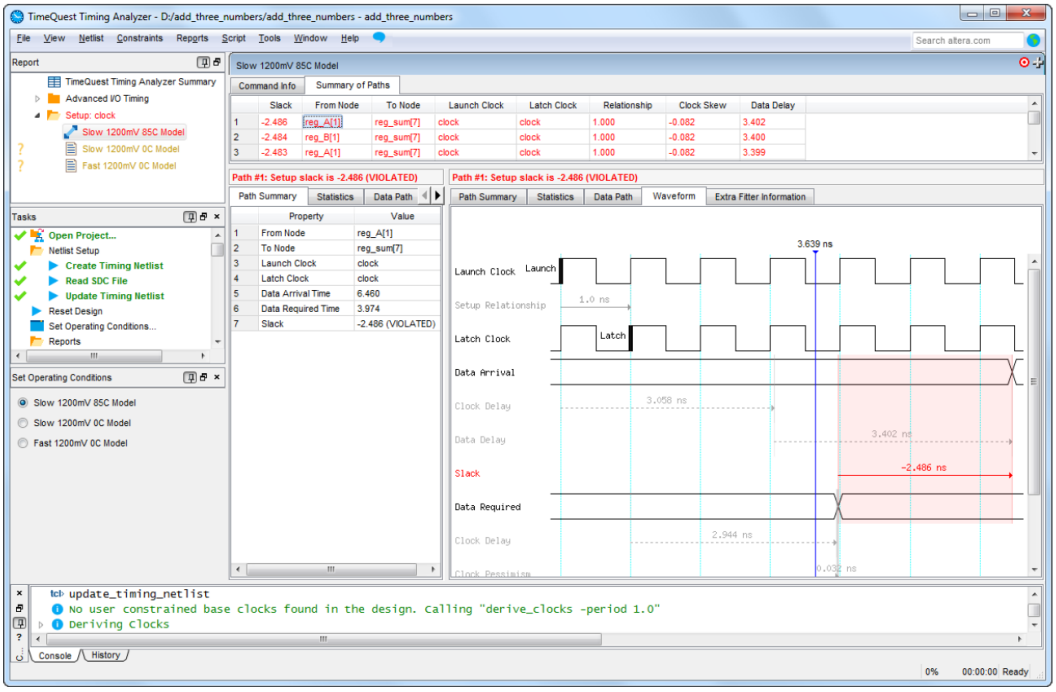


Figure 3A: The TimeQuest GUI.

The TimeQuest GUI consists of several sections. They include the main menu at the top, the Report pane in the top-left corner, the Tasks pane on the left, the detailed results panes in the middle, and the Console display at the bottom of the window. The main menu is used to interact with the TimeQuest tool and issue commands. The Report pane contains any reports generated when using the tool, and the Tasks pane contains a sequence of actions that can be performed to obtain timing reports. The View pane hosts any windows that are opened, such as details about the timing information. The Console window at the bottom provides access to a command line for TimeQuest.

We will focus on the panes in the middle of the TimeQuest GUI, which show detailed results of the timing analysis. The Slow 1200mV 85C Model pane lists the analyzed paths in the circuit from source to destination flip-flops. The first column in this pane shows the Slack of each path with respect to the (default) clock constraint of 1 ns. For each path in the circuit, the slack value represents the difference between the clock period constraint and the path delay:

- a positive slack means that the delay is smaller than the constraint,
- a negative slack represents a delay that is larger than the constraint.

If the slack values in the report are negative, they are shown in red, because the timing results fail to meet the required constraint. If the maximum negative slack value is -2.486 ns, it means that the worst-case delay path is 1 - (-2.486) = 3.486 ns long. This corresponds to a maximum usable clock frequency, F_{max} , of about 286.86 MHz. Note: this is just a first estimation, thus to find the real maximum usable clock frequency you have to repeat the implementation with the new timing constraint and check the corresponding slack.

The waveforms shown for path #1 in Figure 3A illustrate the detailed timing situation. The waveforms show that the clock signal takes 3.058 ns to propagate from its input pin to the source flip-flop, and

then this flip-flop produces data that takes 3.402 ns to reach the destination flip-flop. Also, the clock signal takes 2.944 ns to reach the destination flipflop. The clock1 delays at the source and destination flip-flops represent a clock skew t_{skew} = 2.944 - 3.058 = - 0.114 ns.
A value of 0.032 to account for *Clock Pessimism* is added to the clock skew, making the final clock skew value to be - 0.082 ns. The difference in the required arrival time of the data on this path and the actual arrival time is shown by the negative slack value of 1- 3.402 + t_{skew} = - 2.484 ns. TimeQuest adds a value of - 0.002 ns to account for *Clock Uncertainty*, leading to the final slack value of -2.486 ns.

Setting Up Timing Constraints for a Design

In the TimeQuest GUI, select **`Constraints -> Create Clock`**, which leads to the *Create Clock* window shown in Figure 4A. Set the Clock name to *clock* and the Period to 4.000 ns.

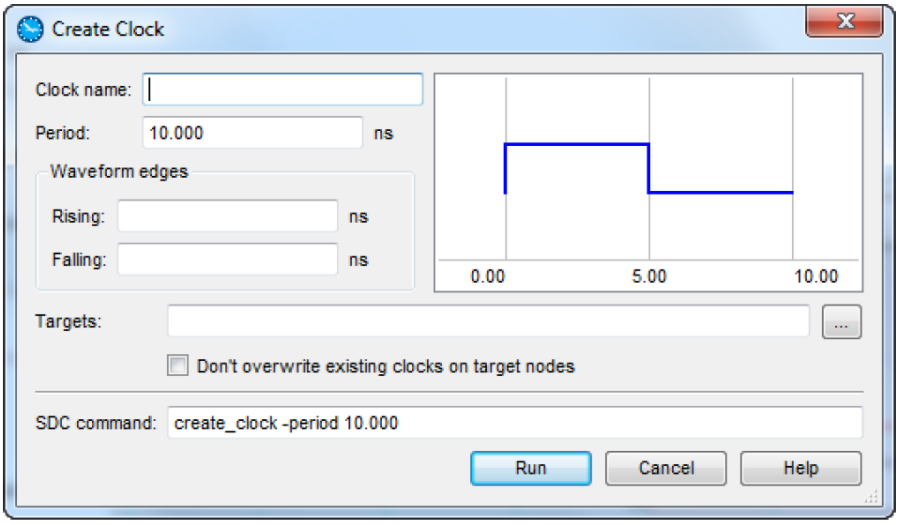


Figure 4A: The Create Clock window.

It is necessary to tell TimeQuest which signal in our design this clock constraint applies to. To do this, click the **...** button to the right of the Targets field, leading to the Name Finder window shown in Figure 5. Click List to show all of the ports in the design. In the list of ports, highlight *clock*, which is the clock signal in our circuit, press **`>,`** then click OK. Finally, click the **`Run`** button in the Create Clock window to apply the constraint.

In order to use this clock constraint for future compilations and timing analysis of this project, we must save the constraint to a file of the type *sdc*, which stands for Synopsys Design Constraint. This file uses an industry standard format for specifying timing constraints. Select **`Constraints -> Write SDC File...`** to write all of the currently set constraints (in our case just the one clock constraint) to an SDC file. This leads to a dialog window, where we specify the `sdc` file name, e.g. *add_three_numbers.sdc*. Note that Quartus will by default try to locate and use the sdc file whose file name matches the project name (except for the .sdc extension). However, it is recommended to add it as file of the project (`Project -> Add/Remove Files in Project`).

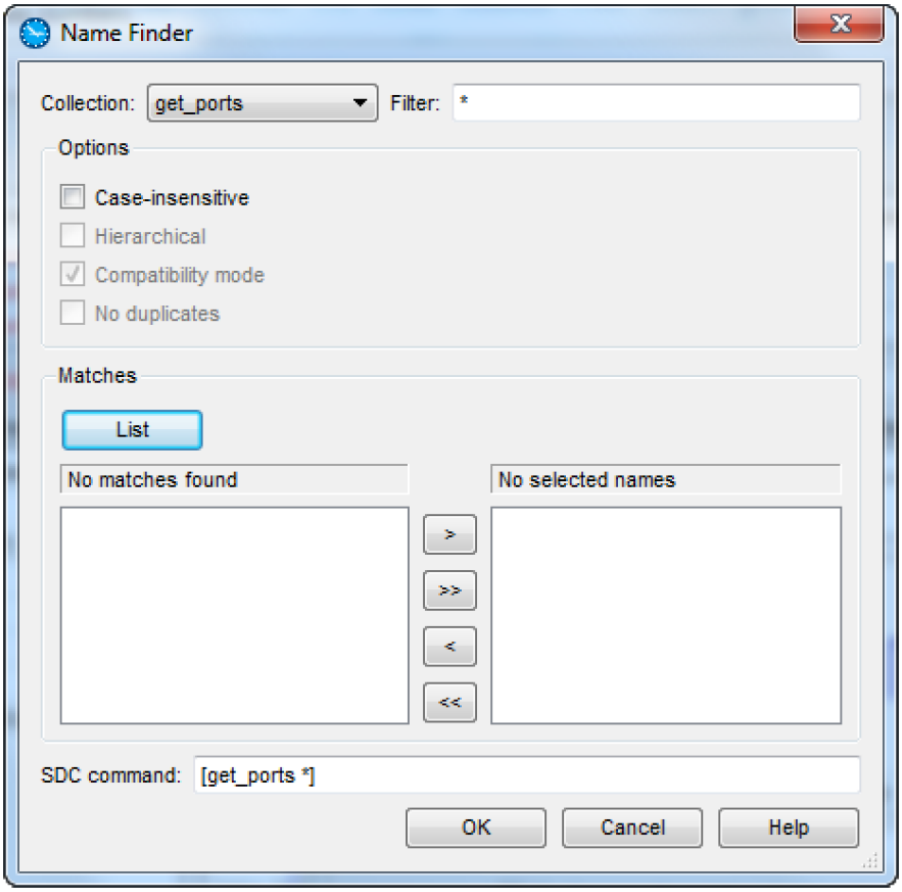


Figure 5A: The Name Finder window.

You will notice that our clock timing report Setup clock is now out of date, as indicated by the yellow font and highlighting. Right-click the report and select Regenerate, to re-run the timing analysis using the new 4 ns clock period constraint. This analysis results in a positive slack of 0.514 ns.

Now, close the TimeQuest GUI. Since we have not recompiled the example design after adding the 4 ns clock period constraint, the timing analysis is based of the circuit that was produced by Quartus Prime when using the (default) 1 ns clock period constraint. To see the effect of the new timing constraint on the compilation results, recompile the project. Then, follow the steps described previously to perform a new timing analysis using TimeQuest. The 4 ns timing constraint will cause the Quartus Prime optimization algorithms to make different decisions from those made when the (default) 1 ns constraint was used. In particular, the optimization algorithms will likely take less time to execute, because once the generated circuit has sufficient positive slack to meet the constraint, the algorithms can terminate. Figure 6A shows the results of timing analysis, with a positive slack of 0.600 ns.

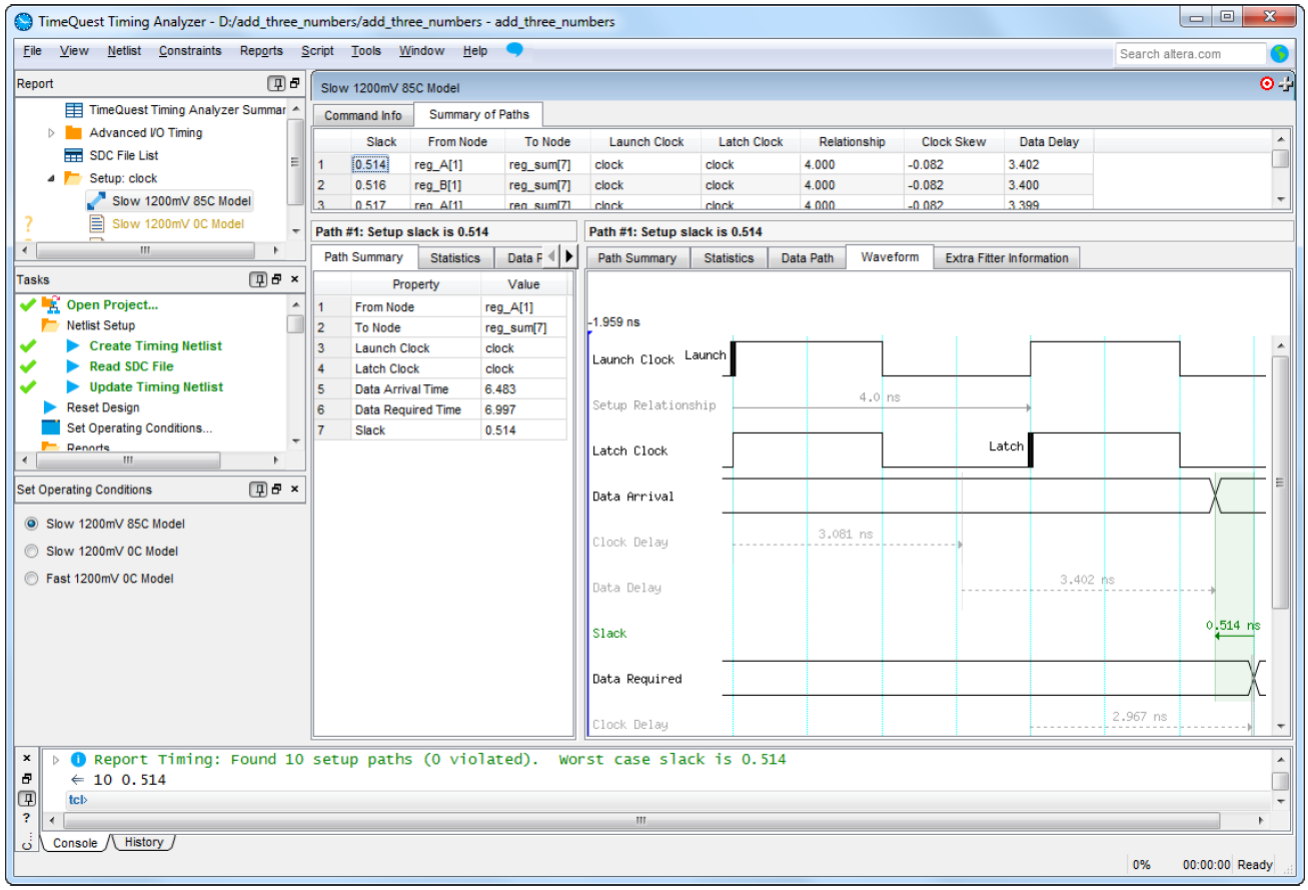


Figure 6A: Updated waveforms.

The TimeQuest Graphical User Interface

In the previous example, we accessed TimeQuest via the Quartus Prime Compilation Report. Another way to open the TimeQuest GUI is to use the command **`Tools > TimeQuest Timing Analyzer`** from the main Quartus Prime window.

There are several commands available in the TimeQuest GUI to obtain timing data.

- In the Tasks pane, we can create a timing netlist, which will be used to perform the analysis, by double-clicking the *Create Timing Netlist* command. Note that while this netlist was generated automatically when performing a timing analysis, as described in the previous sections, the netlist can also be generated manually by using the Tasks pane.
- By double-clicking *Read SDC File,* we instruct the analyzer to read a Synopsys Design Constraints (SDC) file and apply the constraints during analysis. The SDC file can be editedmanually (using any text editor) at any time, and the timing analysis can then be re-run using the new constraints.

- By double-clicking the *Update Timing Netlist* command, we use the specified constraints to determine which parts of the circuit fail to meet them. Once the timing netlist is updated, reports can be generated.
- To generate a report, double-click on a report name in the Tasks pane e.g. the *Report Setup Summary*. By right-clicking on the clock name and then selecting *Report Timing...,* we open the *Report Timing dialog* previously shown in Figure 2A. Several fields in this window help specify the data to be reported.
    - The first field is the **`Clocks`** field, which specifies the types of paths that will be reported. More precisely, it specifies the clock signal at the source flip-flops (*From clock*) and the clock signal at the destination flip-flops (*To clock*). This will limit the reporting to the register-to-register paths only.
    - The next field is the **`Targets`** field. It can be used to refine the report by focusing only on certain paths in the design. As an example, we can specify the starting and the ending point of the paths of interest by filling the *From* and *To* fields. In addition, we can look at only the paths that pass through certain nodes in the design.
    - The next two fields are the **`Analysis type`** and **`Paths`** fields. The Analysis type field specifies if the report should contain setup, hold, recovery, or removal information. Each of these analyses looks for distinct timing characteristics in your design. For example, the setup analysis determines if the data arrives at a flip-flop sufficiently early for the flip-flop to store it reliably, given a clock period. On the other hand, the hold analysis determines if the data input at any given flip-flop remains stable after the positive edge of the clock long enough for the data to be stored in a flip-flop reliably. The Paths field specifies the maximum number of paths to be reported and the maximum slack required for a path to be included in the report.
    - The next set of fields specify the **`Output`** format and the level of detail in the report. The output could be to a window or a file.
    - The last field is the **`Tcl command`** field. This field shows a command that will be executed to generate the requested report.
- Timing constraints can be entered by using the Constraints menu in the TimeQuest GUI. To assign a clock constraint, select **`Create Clock...`** from the Constraint menu.
- Once the constraints file is saved, it can be used by Quartus Prime when compiling a project. This is done in Quartus Prime by going into **`Assignments > Settings... > TimeQuest Timing Analyzer`**, and adding the SDC file to the TimeQuest timing analyzer settings. The Quartus Prime project can then be recompiled to use the constraint.

TimeQuest is capable of analyzing circuits that contain multiple clocks. In designs with multiple clocks, it is important to apply constraints to each clock before performing timing analysis. Doing so will make the analyzer provide the same reports as described in previous sections.

Appendix B – Power consumption estimation with the Power Analyzer tool

This section explains how to estimate the power consumption of your circuit using the Power Analyzer too.

Power consumption in an electronic circuit is made mainly of two contributions: static and dynamic power consumption. Static power consumption depends on the technology and is mainly due to leakage currents. In moder FPGAs it is a relevant part of the total power consumption. Dynamic power consumption instead depends on the fact that the circuit is switching and it can be divided into two main contributions: charging/discharging load capacitances (switching power) and short-circuit currents flowing from the power supply to the ground during circuit switching (short-circuit power). As a consequence, dynamic power consumption depends on the switching activity of the circuit. A circuit which is switching frequently is also charging/discharging frequently load capacitances and is generating frequently short-circuits between power supply and ground, indeed.
It is well known that dynamic power consumption can be indeed written as:

$$P_{dynamic} = \left[\frac{1}{2} C V^2 + Q_{ShortCircuit} V \right] f \cdot activity$$

Capacitance Charging — Short Circuit Charge During Switching — Percent of Circuit that Switches Each Cycle

where V is the power supply voltage and f is the clock frequency.
Once the circuit is implemented, Quartus has all the required information to evaluate the dynamical power consumption, except the activity. Thus, in order to estimate the power consumption of the circuit we must provide Quartus with the activity information. A simple way to obtain the activity information is via simulation. Indeed, by simulating the netlist of the implemented circuit we can obtain an accurate estimation of the activity of each node.

Getting the activity via simulation

1. When implementing a circuit with Quartus, we create a project and we obtain as a result several files and folders. In the folder contining the Quartus project file (.qpf file), there is a folder, which default name is **`simulation`**. Depending of the project settings, this folder can contain another folder which name is the associated simulator. Thus, if during the project creation you specified a simulator (e.g. ModelSim/QuestaSim) then Quartus will create the proper folder in **`simulation.`** You can check and update the simulator settings via **`Assignments > Settings > EDA Tool Settings`**. In the column named *Tool Type* find *Simulation* and select *Questa Intel FPGA* in the column *Tool Name* and *Verilog HDL* in the column *Format(s).* Then press *OK* (see Figure 1).
   If you update these parameters, in the main window of Quartus you will see that the item *EDA Netlist Writer* in the *Compile Design* Task has yellow question mark instead of a green tick. Right click *EDA Netlist Writer* and click *Start* (see Figure 2). With these settings, Quartus

generates a Verilog netlist for your design in the folder **`simulation > questa`**. The netlist file has extension .vo

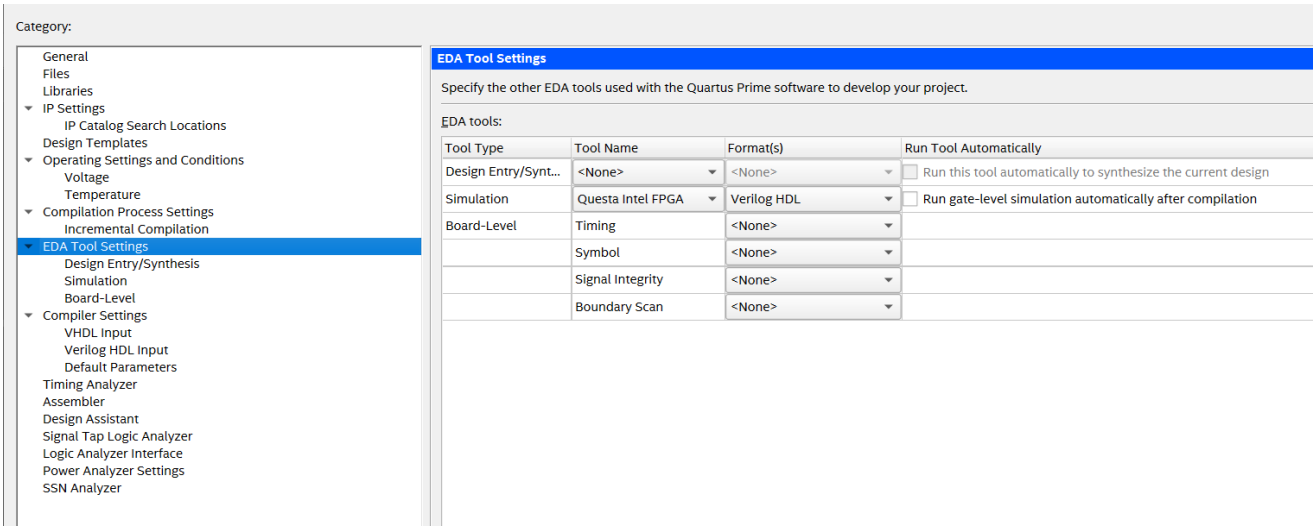


Figure 1 EDA Tool Settings.

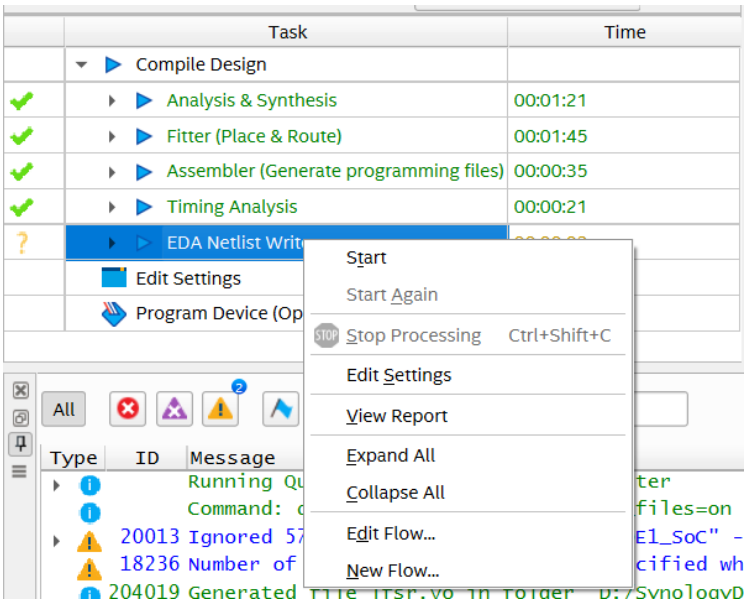


Figure 2 Start EDA Netlist writer

2. Assuming that you already have a testbench stimulating your circuit in realistic conditions, you can create a ModelSim/QuestaSim project including the netlist (.vo file) and your testbench. Note: the testbench can be either in Verilog or in VHDL. Once the project has been set up you can compile the files as in RTL projects. Then, to properly start the simulation we have to specify some supplementary options to ModelSim/QuestaSim. To be sure that the simulator time resolution is compatible with the one of netlist we must force it to ps (`-t ps`). As the netlist contains FPGA basic cells, we have to include simulation models of the cells. The

models are placed in the installation folder of Quartus in **<quartus_installation_folder>/questa_fse/intel/verilog** the two sets of models we need to perform the simulation are *altera* and *cyclonev*. They can be included in the simulation by forcing ModelSim/QuestaSim to link them as

```
-L <quartus_installation_folder>/questa_fse/intel/Verilog/altera
-L <quartus_installation_folder>/questa_fse/intel/Verilog/cyclonev
```

A simple way to start the simulation with proper parameters is exploiting the simulator command line. Assuming that the top of hierarchy is named *tb_lfsr*, then the command line becomes:

```
vsim -t ps -voptargs=+acc work.tb_lfsr \
-L <quartus_installation_folder>/questa_fse/intel/verilog/altera \
-L <quartus_installation_folder>/questa_fse/intel/verilog/cyclone
```

Once the project is loaded and ready to be simulated, we open a file to store the activity. The activity is recorded in a text format called value change dump (vcd) format. This is obtained by issuing in the simulator command line the following commands:

```
vcd file <activity_file_name>.vcd
vcd add /tb_lfsr/uut/*
```

The first command creates an empty vcd file ready to be filled with the circuit activity. The second command tells the simulator which signals have to be monitored. In this example we assume that the project hierarchy is *tb_lfsr* (the testbench) which instantiates the component we want to monitor. The instance of the component is labeled as *uut*. The command tells the simulator that we want to monitor all signals inside instance *uut*, which represent indeed the activity of *uut*. Now we can run the simulation for the proper amount of time, e.g.

```
run 100 ms
```

Most of the time the simulator is retaining the activity internally. To force it to dump all the activity in the vcd file, we have to stop the simulation, e.g.

```
quit -sim
```

3. Now we come back to Quartus and start the power estimation tool: **Processing > Power Analyzer Tool**. This opens the window shown in Figure 3, where the first part regards the activity input file. By pressing the button *Add Power Input File(s)*, the window shown in Figure 4 pops up. Check "Use input file(s) to initializer toggle rates …"; then, click *Add*. A small window opens: click the three dots in the *File Name* field and select the .vcd file generated with ModelSim/QuestaSim. Check that the *Entity* name refers to is the component you are analyzing. Check that *VCD file* is selected. Press OK. You are now back to the window shown in Figure 4 where the field Input File(s) now contains the information you provided with the .vcd file. Click OK. You are now back to the window shown in Figure 3. Scroll down in the *Power*

Analyzer Tool tab and press Start. The power analyzer starts the analysis. When finished, in the *Compilation Report* tab you will see the item Power Analyzer. Click next to the folder icon to open the menu and select *summary*, as shown in Figure 5. In the *Power Analyzer Summary* tab you have the result of the power analysis.

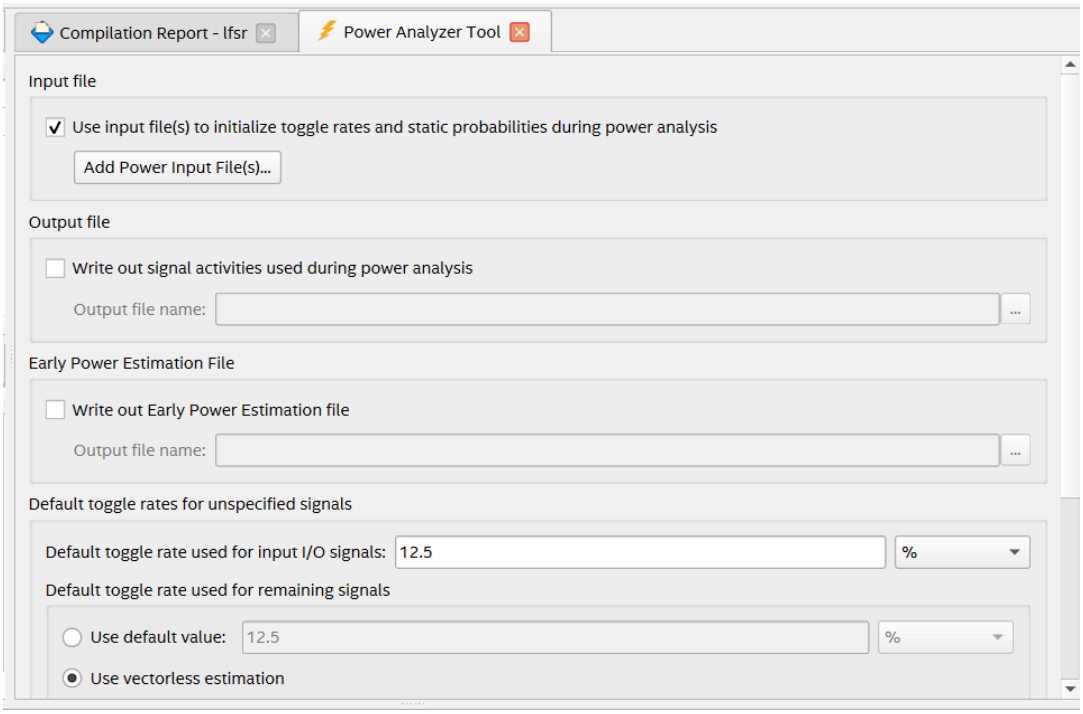


Figure 3 Power Analyzer

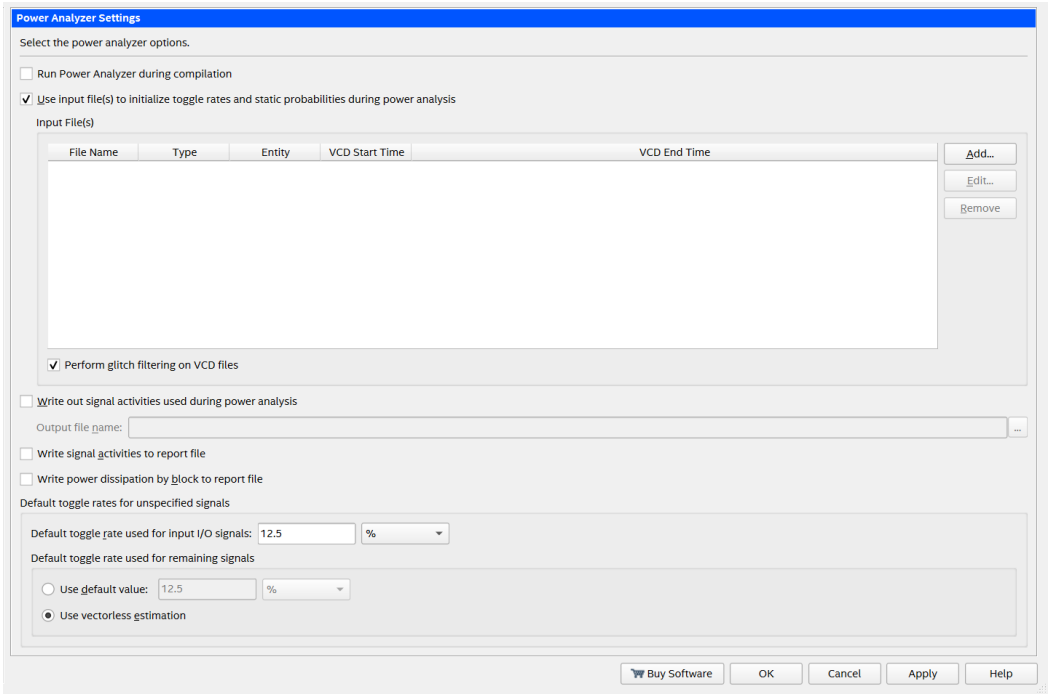


Figure 4 Power Analyzer Settings

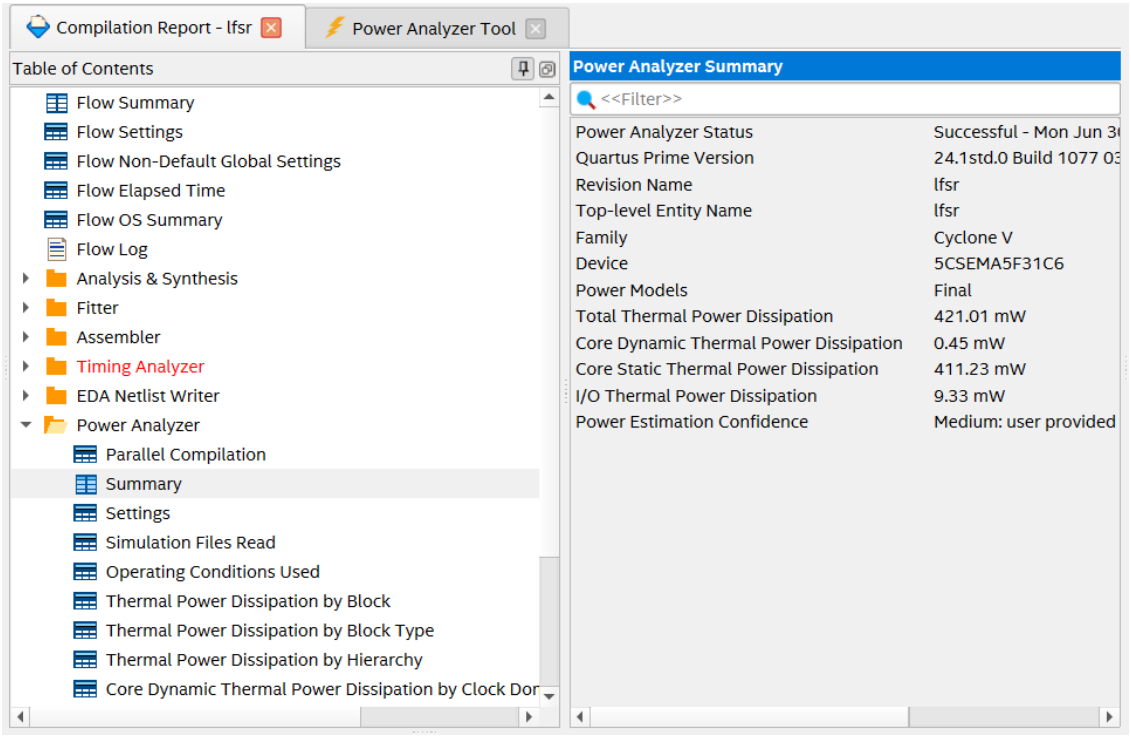


Figure 5 Power Analyzer Summary